

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1205	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL1- ANALYTICAL PROBLEM SOLVING
Weightage:	45%	Total Marks:	25
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:	THIRUMALAPUDI PRANAVI	Reg.No.:	192411222
Department:	CSE -BE	Date:	28-07-26

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and -23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.
2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look-ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.
3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, -6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.
4. A processor is required to perform division of two binary numbers (e.g., $13 \div 3$) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.
5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., -13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

Question 1

Arithmetic Operations on Signed 8-bit Integers Using 2's Complement Representation

Aim

To analyze how a processor performs addition and subtraction on signed 8-bit integers using 2's complement representation, detect overflow, and interpret the final result.

Theory

Computers represent signed integers using 2's complement representation because it simplifies arithmetic operations. In an 8-bit system:

- Total bits = 8
- Most Significant Bit (MSB) is the sign bit
 - 0 → Positive Number
 - 1 → Negative Number

The range of an 8-bit signed integer is
−128 to +127

Advantages of 2's Complement:

- Only one representation for zero.
- Simplifies addition and subtraction.
- Hardware implementation is simple.
- Overflow detection becomes easy.

Given Data

- Number 1 = +45
- Number 2 = −23

Step 1: Convert +45 into Binary

Divide by 2 repeatedly.

Division	Quotient	Remainder
$45 \div 2$	22	1
$22 \div 2$	11	0
$11 \div 2$	5	1
$5 \div 2$	2	1
$2 \div 2$	1	0
$1 \div 2$	0	1

Reading the remainders from bottom to top:

101101

Convert to 8 bits:

00101101

Therefore,

+45 = 00101101

Step 2: Convert -23 into Binary

Convert +23 first

23 in binary:

00010111

Find 1's Complement

Invert every bit.

00010111

↓↓↓↓↓↓↓↓

11101000

Find 2's Complement

Add 1.

11101000

+00000001

11101001

Therefore,

-23 = 11101001

Step 3: Perform Addition

Operation:

+45 + (-23)

Write both numbers.

00101101

+ 11101001

1 00010110

The carry generated beyond 8 bits is discarded.

Result:

00010110

Step 4: Convert Result to Decimal

00010110

=16+4+2

=22

Hence,

$$45 + (-23) = 22$$

Step 5: Check Overflow

Overflow occurs only when:

Case 1

Positive + Positive = Negative

Example:

$$100 + 50$$

may exceed +127.

Case 2

Negative + Negative = Positive

Example:

$$-80 + (-70)$$

may become positive because of overflow.

Here,

Positive + Negative

The result is positive and lies within the range

-128 to +127

Therefore,

No Overflow Occurs

Step 6: Processor Overflow Detection

The processor compares

- Carry into MSB
- Carry out of MSB

Formula:

$$\text{Overflow} = \text{Carry into MSB} \text{ XOR } \text{Carry out of MSB}$$

If

$$\text{Carry In} = \text{Carry Out}$$

then

$$\text{Overflow} = 0$$

If

$$\text{Carry In} \neq \text{Carry Out}$$

then

$$\text{Overflow} = 1$$

For this example,

$$\text{Overflow Flag} = 0$$

Subtraction

Operation

$$45 - (-23)$$

Subtracting a negative number is equivalent to adding the positive number.

$$45 - (-23)$$

=

$$45 + 23$$

Binary values

$$45 = 00101101$$

$$23 = 00010111$$

Addition

$$\begin{array}{r} 00101101 \\ + 00010111 \\ \hline \end{array}$$

$$01000100$$

Result

$$01000100$$

Convert Result into Decimal

$$01000100$$

$$= 64 + 4$$

$$= 68$$

Hence,

$$45 - (-23) = 68$$

Overflow Analysis

Both operands are positive.

Result is also positive.

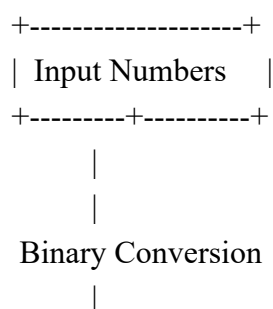
68 lies within

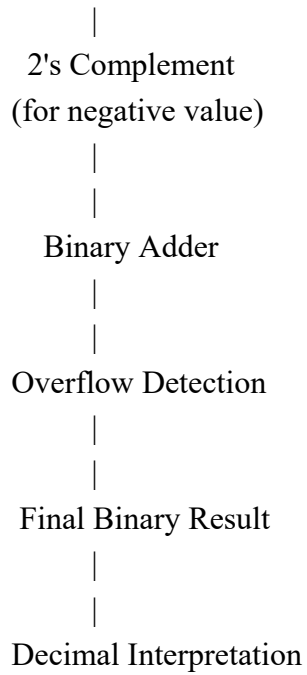
-128 to +127

Hence,

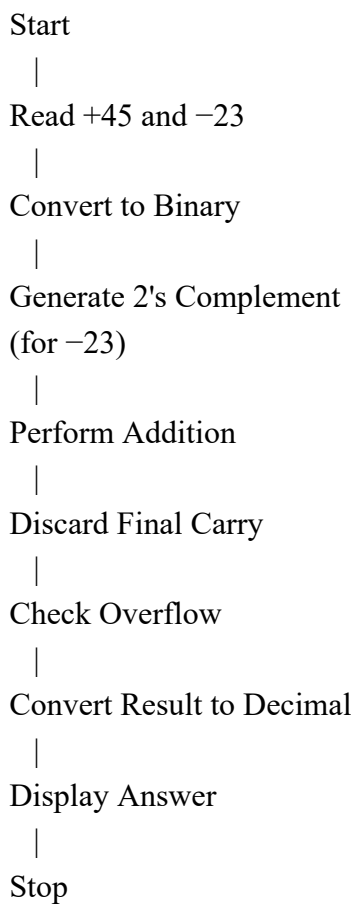
Overflow does not occur.

Processor Block Diagram





Flowchart



Observation

Operation	Binary Result	Decimal Result	Overflow
$45 + (-23)$	00010110	22	No
$45 - (-23)$	01000100	68	No

Advantages of 2's Complement

1. Easy implementation in hardware.
2. Same circuit performs addition and subtraction.
3. Only one representation of zero.
4. Simple overflow detection.
5. Faster arithmetic operations.

Applications

- Arithmetic Logic Unit (ALU)
- Digital Signal Processing
- Embedded Systems
- Microprocessors
- Computer Arithmetic Units

Conclusion

The processor successfully performs signed addition and subtraction using 2's complement representation. The numbers +45 and -23 are converted into binary, the required arithmetic operation is performed, and the carry beyond the eighth bit is discarded. The processor checks the carry into and out of the Most Significant Bit (MSB) to detect overflow. Since the result remains within the valid 8-bit signed integer range (-128 to +127), no overflow occurs. This demonstrates that 2's complement arithmetic provides an efficient, reliable, and hardware-friendly method for performing signed integer operations.

assessments.

Question 2

A High-Performance Computing System Replaces a Ripple Carry Adder with a 4-bit Carry Look-Ahead Adder (CLA)

Aim

To analyze the working of a **4-bit Carry Look-Ahead Adder (CLA)** by generating propagate and generate signals, computing carries in advance, comparing its performance with a Ripple Carry Adder (RCA), and explaining why CLA is preferred in modern processors.

Introduction

Addition is one of the most frequently performed operations in a processor. Every arithmetic operation such as multiplication, division, address calculation, and data processing requires addition.

A **Ripple Carry Adder (RCA)** performs addition by passing the carry from one stage to the next. Since each bit must wait for the previous carry, the computation becomes slow as the number of bits increases.

To overcome this delay, processors use a **Carry Look-Ahead Adder (CLA)**. The CLA predicts carry values in advance using special logic circuits, making addition much faster.

Theory

A Carry Look-Ahead Adder computes the carry bits using two signals:

1. Generate Signal (G)

A carry is generated when both input bits are 1.

Formula

$$G_i = A_i \times B_i$$

2. Propagate Signal (P)

A carry is propagated when either one of the input bits is 1.

Formula

$$P_i = A_i \oplus B_i$$

where

- $\times$ = AND operation
- $\oplus$ = XOR operation

Given Inputs

Let

$$A = 1010$$

$$B = 0110$$

Assume

$$\text{Carry Input (C0)} = 0$$

Step 1: Write the Binary Numbers

A = 1 0 1 0

B = 0 1 1 0

Bit positions

Bit A B

A3 1 0

A2 0 1

A1 1 1

A0 0 0

Step 2: Calculate Generate (G)

Formula

[
G=A\times B
]

Bit	Calculation	Generate (G)
0	0\times 0	0
1	1\times 1	1
2	0\times 1	0
3	1\times 0	0

Therefore,

G3 G2 G1 G0

0 0 1 0

Step 3: Calculate Propagate (P)

Formula

[
P=A\oplus B
]

Bit	Calculation	Propagate (P)
0	0\oplus 0	0
1	1\oplus 1	0
2	0\oplus 1	1
3	1\oplus 0	1

Therefore,

P3 P2 P1 P0

1 1 0 0

Step 4: Calculate Carry Signals

Assume

$$C_0 = 0$$

Carry C

Formula

$$[C_1 = G_0 + P_0 C_0]$$

Substitute values

$$C_1 = 0 + (0 \times 0)$$

$$C_1 = 0$$

Carry C2

Formula

$$[C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0]$$

Substitute

$$= 1 + (0 \times 0) + (0 \times 0 \times 0)$$

$$= 1$$

Therefore

$$C_2 = 1$$

Carry C3

Formula

$$[C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0]$$

Substitute

$$= 0 + (1 \times 1) + (1 \times 0 \times 0) + (1 \times 0 \times 0 \times 0)$$

$$= 1$$

Therefore

$$C_3 = 1$$

Carry C4

Formula

$$[C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0]$$

Substitute

$$= 0 + (1 \times 0) + (1 \times 1 \times 1) + (1 \times 1 \times 0 \times 0) + (1 \times 1 \times 0 \times 0 \times 0)$$

$$=1$$

Therefore

$$C_4=1$$

Step 5: Calculate Sum Bits

Formula

$$[S = P \oplus C]$$

S0

$$S_0 = P_0 \oplus C_0$$

$$= 0 \oplus 0$$

$$= 0$$

S1

$$S_1 = P_1 \oplus C_1$$

$$= 0 \oplus 0$$

$$= 0$$

S2

$$S_2 = P_2 \oplus C_2$$

$$= 1 \oplus 1$$

$$= 0$$

S3

$$S_3 = P_3 \oplus C_3$$

$$= 1 \oplus 1$$

$$= 0$$

Final Output

$$\text{Carry} = 1$$

$$\text{Sum} = 0000$$

Final Answer

10000

Decimal Verification

1010 =10

0110 =6

10+6=16

Binary=10000

Correct Answer

Carry Look-Ahead Logic Diagram

A0 -----\
 AND ----- G0
B0 -----/

A0 -----\
 XOR ----- P0
B0 -----/

↓

Carry Generator

C1 C2 C3 C4

↓

Sum Generator

S0 S1 S2 S3

Ripple Carry Adder Working

Bit0

↓

Carry

↓

Bit1

↓

Carry

↓

Bit2

↓

Carry

↓

Bit3

Each Full Adder must wait for the previous carry.

This increases delay.

Carry Look-Ahead Adder Working

Generate Signals

↓

Propagate Signals

↓

Carry Generator

↓

All Carries Generated Together

↓

Sum Generator

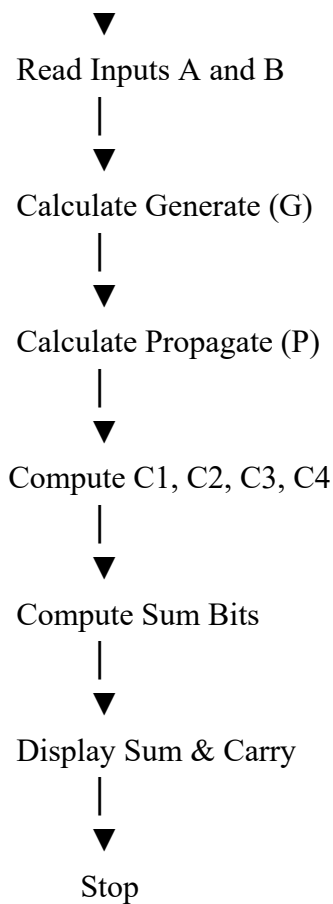
All carries are produced simultaneously.

No waiting is required.

Flowchart

Start

|



Comparison Between Ripple Carry Adder and Carry Look-Ahead Adder

Feature	Ripple Carry Adder	Carry Look-Ahead Adder
Carry Generation	Sequential	Parallel
Speed	Slow	Very Fast
Propagation Delay	High	Very Low
Hardware Complexity	Simple	Complex
Power Consumption	Low	Slightly Higher
Cost	Low	Higher
Performance	Moderate	Excellent
Used In	Small Circuits	Modern CPUs and ALUs

Why CLA Improves Speed

1. Carries are generated simultaneously instead of sequentially.
2. Eliminates ripple carry delay.
3. Reduces processor execution time.
4. Improves Arithmetic Logic Unit (ALU) performance.
5. Suitable for high-speed processors.
6. Supports pipelined processor architecture.
7. Faster arithmetic increases overall system performance.

Advantages of Carry Look-Ahead Adder

- High-speed addition.
- Reduces carry propagation delay.
- Suitable for processors with large word sizes.
- Improves CPU performance.
- Efficient in digital signal processing.
- Widely used in modern microprocessors.

Disadvantages

- Requires more hardware.
- More complex circuit design.
- Higher power consumption than Ripple Carry Adder.
- Costlier to implement.

Applications

- Arithmetic Logic Unit (ALU)
- Intel and AMD processors
- Digital Signal Processors (DSP)
- Graphics Processing Units (GPU)
- Supercomputers
- High-performance computing systems
- Embedded systems
- Scientific calculators

Observation

- Ripple Carry Adder computes carries one after another, resulting in higher delay.
- Carry Look-Ahead Adder computes all carries in parallel, significantly reducing computation time.
- Although CLA requires more hardware, its speed advantage makes it ideal for modern processors.

Conclusion

The **4-bit Carry Look-Ahead Adder (CLA)** significantly improves addition speed by generating **propagate (P)** and **generate (G)** signals and computing all carry values simultaneously. Unlike the Ripple Carry Adder, which waits for each carry to propagate through every stage, the CLA eliminates sequential carry delay, making arithmetic operations much faster. Because of its low propagation delay and high performance, the Carry Look-Ahead Adder is widely used in **modern processors, ALUs, digital signal processors, and high-performance computing systems.**

Question 3

Signed Multiplication Using Booth's Algorithm ($-6 \times +5$)

Aim

To analyze the multiplication of two signed binary numbers (-6 and $+5$) using Booth's Algorithm, explain the role of the Accumulator (A), Multiplier (Q), Multiplicand (M), and Booth Bit (Q-1), and show how arithmetic right shifts reduce the number of addition and subtraction operations.

Introduction

Multiplication is one of the most important arithmetic operations performed by a processor. A simple binary multiplication method requires repeated addition, which becomes slow when multiplying signed numbers.

Booth's Algorithm is an efficient multiplication algorithm used in processors because:

- It directly supports signed numbers using 2's complement.
- It reduces the number of addition and subtraction operations.
- It is faster than conventional binary multiplication.
- It is widely used in CPUs, DSPs, and microprocessors.

Theory:

Booth's Algorithm examines two bits at a time:

- Q_0 = Least Significant Bit (LSB) of the multiplier.
- $Q-1$ = Booth Bit (initially 0).

The algorithm performs operations according to the following rules:

Q_0 $Q-1$	Operation
00	No operation
11	No operation
01	Add Multiplicand (M)
10	Subtract Multiplicand (M)

After every operation, an Arithmetic Right Shift (ARS) is performed.

Given Data

Multiplicand (M) = -6

Multiplier (Q) = $+5$

Step 1: Convert Numbers into Binary

Convert $+5$

$+5 = 0101$

Convert -6

First convert $+6$

$+6 = 0110$

Find 1's complement

0110

↓↓↓↓
 1001
 Add 1
 1001
 +0001

 1010
 Therefore,
 $-6 = 1010$

Step 2: Initialize Registers

Register	Value
Accumulator (A)	0000
Multiplicand (M)	1010
Multiplier (Q)	0101
Booth Bit (Q-1)	0

Initial Register

A Q Q-1

0000 0101 0

Step 3: Perform Booth's Algorithm

The number of iterations equals the number of bits.

Here,

4-bit multiplication $\rightarrow$ 4 iterations

Iteration 1

Check Q_0Q_{-1}

$Q_0 = 1$

$Q_{-1} = 0$

10

According to Booth's Rule

$10 \rightarrow A = A - M$

Since

$A = 0000$

$M = 1010$

Subtract M

0000

−1010

=

0110

New A

0110

Current Register

A Q Q-1

0110 0101 0

Arithmetic Right Shift

Before Shift

0110 0101 0

After Shift

0011 0010 1

Iteration 2

Check

$Q_0=0$

$Q-1=1$

01

Rule

01 → Add M

Perform

0011

+

1010

1101

Current Register

A

1101

Arithmetic Right Shift

Before Shift

1101 0010 1

After Shift

1110 1001 0

Iteration 3

Check

$Q_0=1$

$Q_{-1}=0$

10

Rule

Subtract M

Perform

1110

-1010

=

0100

Current Register

0100

Arithmetic Right Shift

Before Shift

0100 1001 0

After Shift

0010 0100 1

Iteration 4

Check

$Q_0=0$

$Q_{-1}=1$

01

Rule

Add M

Perform

0010

+

1010

1100

Current Register

1100

Arithmetic Right Shift

Before Shift

1100 0100 1

After Shift

1110 0010 0

Final Product

Combine A and Q

A =1110

Q =0010

Result

11100010

Step 4: Convert Result to Decimal

Binary

11100010

Since MSB = 1,

The number is negative.

Find 2's complement.

1's Complement

11100010

↓↓↓↓↓↓↓↓

00011101

Add 1

00011101

+00000001

Decimal
00011110

$=30$

$$11100010 = -30$$
$$-6 \times +5 = -30$$

Iteration	QoQ-1	Operation	Result
1	10	Subtract M	ARS
2	01	Add M	ARS
3	10	Subtract M	ARS
4	01	Add M	ARS

An Arithmetic Right Shift preserves the sign bit.

Before Shift

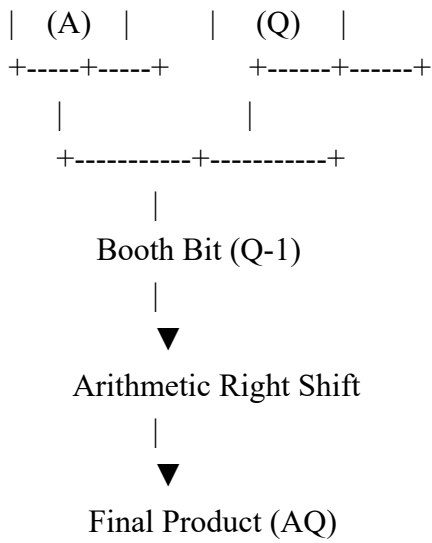
After Shift

The leftmost bit (sign bit) is copied to maintain the correct sign of the number.

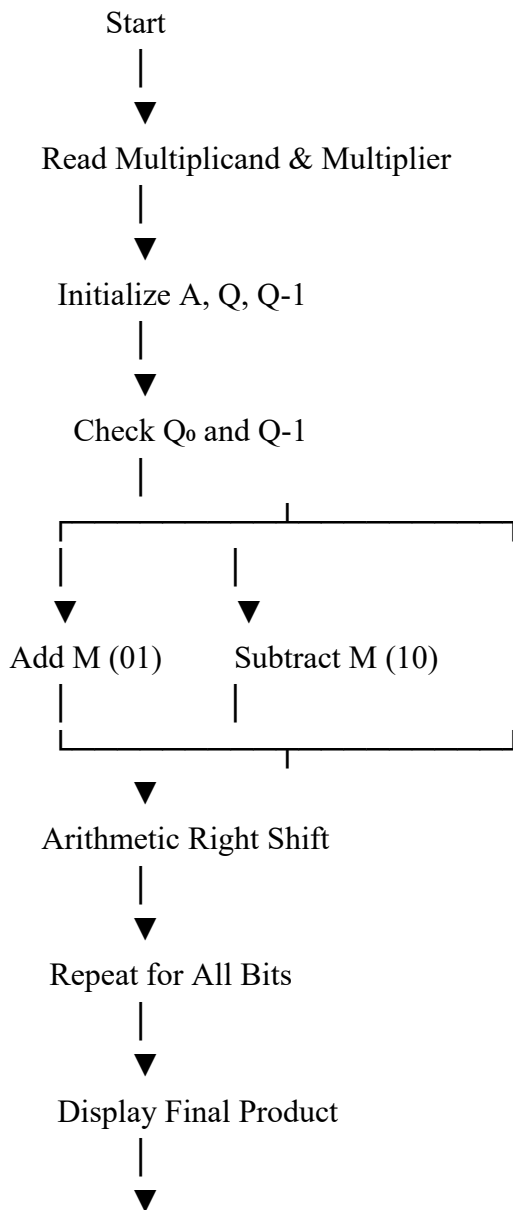
Diagram illustrating the internal structure of the ALU, showing the flow of data between the Multiplicand M, Arithmetic Unit, Accumulator, and Multiplier.

```

graph TD
    M[Multiplicand M] --> AU[Arithmetic Unit]
    AU --> Acc[Accumulator]
    AU --> Mult[Multiplier]
    Acc --> AU
    Mult --> AU
  
```



Flowchart



Stop

Advantages of Booth's Algorithm

1. Supports signed binary multiplication.
2. Reduces the number of addition and subtraction operations.
3. Faster than ordinary binary multiplication.
4. Efficient for numbers containing consecutive 1s.
5. Widely used in modern processors.
6. Requires fewer arithmetic operations.

Disadvantages

1. More complex than ordinary multiplication.
2. Requires additional control logic.
3. Difficult to understand for beginners.
4. Hardware implementation is more complex.

Applications

- Microprocessors
- Arithmetic Logic Unit (ALU)
- Digital Signal Processing (DSP)
- Embedded Systems
- Graphics Processing Units (GPU)
- Scientific Computing
- High-Speed Digital Systems

Observation

- Booth's Algorithm examines Q_0 and Q_{-1} during each iteration.
- It performs addition, subtraction, or no operation based on the bit pair.
- After each operation, an Arithmetic Right Shift is performed.
- This approach minimizes unnecessary additions and improves multiplication speed, especially for signed binary numbers.

Comparison: Booth's Algorithm vs Traditional Binary Multiplication

Feature	Traditional Multiplication	Booth's Algorithm
Signed numbers	Not efficient	Efficient
Addition operations	More	Fewer
Speed	Slower	Faster
Hardware complexity	Simple	Moderate
Processor performance	Lower	Higher
Used in modern CPUs	Rarely	Yes

Conclusion

Booth's Algorithm provides an efficient method for multiplying signed binary numbers by using 2's complement representation and examining the Q_n - Q_{n-1} bit pair in each iteration. It reduces the number of addition and subtraction operations through arithmetic right shifts, making multiplication faster than the traditional binary multiplication method. For the given example, $-6 \times +5$, the final binary product is 11100010, which represents -30 in decimal. Because of its speed and efficient handling of signed numbers, Booth's Algorithm is widely used in modern processors, digital signal processors (DSPs), embedded systems, and Arithmetic Logic Units (ALUs).

Question 4

Division of Binary Numbers ($13 \div 3$) Using Restoring and Non-Restoring Division Algorithms

Aim

To perform the division of two binary numbers ($13 \div 3$) using **Restoring Division** and **Non-Restoring Division** algorithms, analyze each step, compare both methods, and explain why non-restoring division is preferred in modern processors.

Introduction

Division is one of the four basic arithmetic operations performed by a processor. Unlike addition and multiplication, binary division is more complex because the processor must repeatedly subtract the divisor from the dividend while generating the quotient.

There are two commonly used binary division algorithms:

1. **Restoring Division Algorithm**
2. **Non-Restoring Division Algorithm**

The Restoring Division algorithm restores the previous remainder whenever subtraction results in a negative value. In contrast, the Non-Restoring Division algorithm avoids this restoration step, making it faster and more efficient.

Theory

What is Binary Division?

Binary division is similar to decimal division. The processor repeatedly:

- Shifts the dividend.
- Subtracts the divisor.
- Checks whether the remainder is positive or negative.
- Generates one quotient bit at a time.

Given Data

Dividend = 13

Divisor = 3

Convert into Binary

13 = 1101

3 = 0011

Number of bits = 4

Restoring Division Algorithm

Theory

In Restoring Division,

- Shift the registers left.
- Subtract the divisor.
- If the result is negative, restore the previous remainder.
- Store 0 in the quotient.
- Otherwise, keep the remainder and store 1.

Registers Used

Register	Purpose
A	Accumulator (Remainder)
Q	Dividend / Quotient
M	Divisor

Initially,

A = 0000

Q = 1101

M = 0011

n = 4

Algorithm

Step 1

Initialize registers.

A = 0000

Q = 1101

M = 0011

Step 2

Shift left the combined register AQ.

Step 3

Subtract divisor.

$$A = A - M$$

Step 4

If A becomes negative

- Restore A

$$A = A + M$$

- Set quotient bit = 0

Otherwise

- Keep A
- Set quotient bit = 1

Step 5

Repeat until all bits are processed.

Step-by-Step Execution

Initial Values

$$A = 0000$$

$$Q = 1101$$

$$M = 0011$$

Iteration 1

Shift Left

AQ

0000 1101

↓

0001 1010

Subtract divisor

0001

−0011

1110

Negative remainder

Restore

1110

+0011

0001

Quotient bit

0

Iteration 2

Shift Left

0001 1010

↓

0011 0100

Subtract

0011

−0011

0000

Positive

Quotient bit

1

Iteration 3

Shift Left

0000 0100

↓

0000 1000

Subtract

0000

−0011

1101

Negative

Restore

1101

+0011

0000

Quotient bit

0

Iteration 4

Shift Left

0000 1000

↓

0001 0000

Subtract

0001

-0011

1110

Negative

Restore

1110

+0011

0001

Quotient bit

0

Final Result

Quotient = 0100

Remainder = 0001

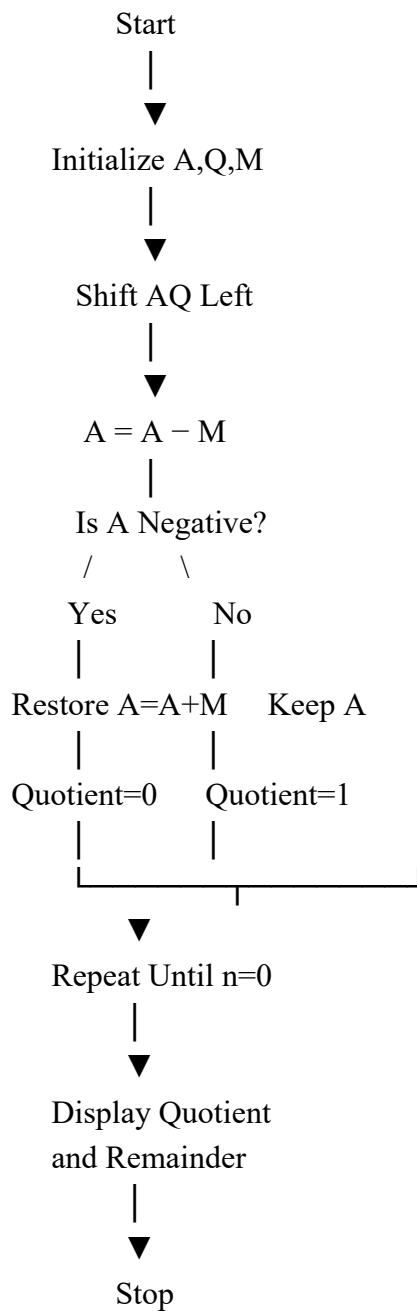
Decimal

$13 \div 3$

= 4

Remainder = 1

Flowchart of Restoring Division



Non-Restoring Division Algorithm

Theory

In Non-Restoring Division,

If the remainder becomes negative,

the processor **does not restore it immediately**.

Instead,

the next iteration performs an addition instead of subtraction.

This eliminates unnecessary restore operations.

Algorithm

Step 1

Initialize registers.

$A = 0000$

$Q = 1101$

$M = 0011$

Step 2

Shift AQ left.

Step 3

If A is positive

$A = A - M$

If A is negative

$A = A + M$

Step 4

Set quotient bit

Positive

$Q_0 = 1$

Negative

$Q_0 = 0$

Step 5

Repeat until all bits finish.

Step-by-Step Execution

Initial

$A = 0000$

$Q = 1101$

$M = 0011$

Iteration 1

Shift Left

Subtract
Negative
Quotient =0
(No Restore)

Iteration 2

Shift Left
Since previous remainder is negative,
Add divisor.
Positive
Quotient =1

Iteration 3

Shift Left
Subtract divisor.
Negative
Quotient =0

Iteration 4

Shift Left
Add divisor.
Positive
Quotient =0

Final Result

Quotient =0100

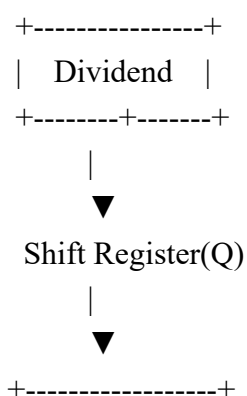
Remainder =0001

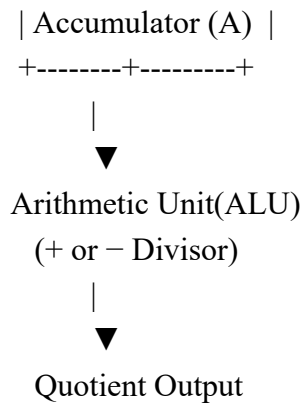
Decimal

4

Remainder 1

Block Diagram





Comparison Between Restoring and Non-Restoring Division

Feature	Restoring Division	Non-Restoring Division
Restore Operation	Required	Not Required
Arithmetic Operations	More	Less
Execution Speed	Slower	Faster
Hardware Complexity	Simple	Slightly Complex
Power Consumption	Higher	Lower
Efficiency	Lower	Higher
Processor Performance	Moderate	Excellent
Used in Modern CPUs	Rarely	Widely Used

Advantages of Restoring Division

1. Easy to understand.
2. Simple hardware implementation.
3. Accurate quotient generation.
4. Suitable for learning binary division.

Disadvantages of Restoring Division

1. Performs unnecessary restore operations.
2. Requires more clock cycles.
3. Slower execution.
4. Lower processor efficiency.

Advantages of Non-Restoring Division

1. Eliminates restore operations.
2. Requires fewer arithmetic operations.
3. Faster execution.
4. Better processor utilization.
5. Suitable for high-speed processors.
6. Reduces execution time.

Disadvantages of Non-Restoring Division

1. Control logic is more complex.
2. Harder to implement than restoring division.
3. More difficult for beginners to understand.

Applications

- Computer processors (CPU)
- Arithmetic Logic Unit (ALU)
- Digital Signal Processing (DSP)
- Embedded systems
- Scientific calculators
- High-performance computing
- Microcontrollers
- Floating-point arithmetic units

Observation

- Both algorithms produce the **same quotient (4)** and **remainder (1)**.
- Restoring Division performs an additional restore operation whenever the remainder becomes negative.
- Non-Restoring Division avoids this extra step by correcting the remainder in the next iteration.
- As a result, Non-Restoring Division executes faster and requires fewer arithmetic operations, making it more suitable for modern processors.

Conclusion

The division of **13 by 3** using both **Restoring** and **Non-Restoring Division** algorithms produces the same result:

- **Quotient = 4 (0100)**
- **Remainder = 1 (0001)**

However, the **Non-Restoring Division Algorithm** is more efficient because it eliminates the restoration step after a negative remainder. This reduces the total number of arithmetic operations, improves execution speed, and enhances processor performance. Due to these advantages, non-restoring division is widely used in **modern CPUs, Arithmetic Logic Units (ALUs), Digital Signal Processors (DSPs), and high-performance computing systems**.

Question 5

IEEE 754 Floating-Point Representation of -13.25 (Single and Double Precision) and Floating-Point Addition

Aim

To analyze the representation of the decimal number -13.25 using the **IEEE 754 floating-point standard** in **single precision (32-bit)** and **double precision (64-bit)** formats, and to explain the process of floating-point addition, including **normalization, rounding, and precision loss**.

Introduction

Many scientific, engineering, and graphics applications require calculations involving **real (fractional) numbers**. Unlike integers, real numbers contain a decimal point, making them difficult to represent exactly in binary.

The **IEEE 754 Standard** is the international standard used by computers to represent floating-point numbers consistently.

It provides:

- Accurate representation of real numbers
- Standard arithmetic operations
- Efficient hardware implementation
- Portability across different computer systems

Most processors today use IEEE 754 format for floating-point arithmetic.

Theory

A floating-point number consists of **three fields**.

1. Sign Bit (S)

- $0 \rightarrow$ Positive number
- $1 \rightarrow$ Negative number

2. Exponent (E)

Stores the exponent using a **bias**.

- Single Precision Bias = **127**
- Double Precision Bias = **1023**

3. Mantissa (Fraction or Significant)

Stores the fractional part after normalization.

The leading **1** is **not stored** because it is implied (hidden bit).

IEEE 754 Formats

Single Precision (32-bit)

Field	Bits
Sign	1
Exponent	8
Mantissa	23

Field	Bits
Total	32 Bits

Double Precision (64-bit)

Field	Bits
Sign	1
Exponent	11
Mantissa	52
Total	64 Bits

Given Number

−13.25

Step 1: Convert Integer Part into Binary

Integer Part

13

Repeated Division by 2

Division	Quotient	Remainder
13 ÷ 2	6	1
6 ÷ 2	3	0
3 ÷ 2	1	1
1 ÷ 2	0	1

Read from bottom to top

13 = 1101

Step 2: Convert Fractional Part into Binary

Fraction

0.25

Multiply repeatedly by 2.

Multiplication Integer Part

$0.25 \times 2 = 0.50$ 0

$0.50 \times 2 = 1.00$ 1

Hence

$0.25 = 0.01$

Step 3: Combine Integer and Fraction

13.25

=

1101.01

Therefore

-13.25

=

-1101.01

Step 4: Normalize the Binary Number

Move the binary point so that only one digit remains before it.

1101.01

↓

1.10101×2^3

Thus,

- Mantissa = **10101**
- Exponent = **3**

Single Precision Representation (32-bit)

Step 5: Find Sign Bit

Since the number is negative

Sign = 1

Step 6: Find Exponent

Actual Exponent

3

Bias

127

Exponent

$3 + 127$

=130

Convert 130 to binary

$130 = 10000010$

Exponent Field

10000010

Step 7: Find Mantissa

Normalized number

1.10101

Remove leading 1.

10101

Fill remaining bits with zeros.

101010000000000000000000

Final IEEE 754 Single Precision Representation

Field	Binary
Sign	1
Exponent	10000010
Mantissa	101010000000000000000000

Final 32-bit Representation

1 10000010 101010000000000000000000

Double Precision Representation (64-bit)

Step 1: Sign Bit

Negative Number

1

Step 2: Exponent

Bias

1023

Exponent

1023+3

=1026

1026 in binary

1000000010

Step 3: Mantissa

1010100

Final IEEE Double Precision Representation

Field	Binary
Sign	1
Exponent	1000000010
Mantissa	1010100

Final Representation

1 1000000010 1010100

Floating-Point Addition

Example

13.25 + 6.5

The processor performs floating-point addition in the following steps.

Step 1: Convert Both Numbers into IEEE Format

13.25

↓

IEEE Representation

6.5

↓

IEEE Representation

Step 2: Compare Exponents

Example

13.25

Exponent =3

6.5

Exponent =2

The exponents are different.

Step 3: Align Exponents

Shift the mantissa of the smaller exponent.

6.5

↓

3.25

↓

Shift Right

↓

Exponent becomes 3

Both numbers now have the same exponent.

Step 4: Add Mantissas

Example

1.10101

+

0.11010

10.01111

Step 5: Normalize Result

The answer is

10.01111

Normalize again.

10.01111

↓

1.001111×2^1

Increase exponent by one.

Step 6: Round the Mantissa

If the mantissa exceeds the available bits,
the processor rounds it according to IEEE 754 rules.

Common rounding mode:

Round to Nearest Even

Step 7: Store Final Result

Store

- Sign
- Exponent
- Mantissa

The floating-point addition is complete.

Flowchart

Start



Read Decimal Number



Convert Integer to Binary



Convert Fraction to Binary



Combine Binary Number



Normalize Number



Find Sign Bit



Calculate Biased Exponent



Store Mantissa



IEEE 754 Representation



Display Result



Stop

Block Diagram

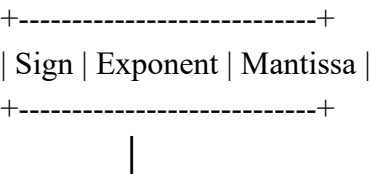
Decimal Number



Binary Converter



Normalization Unit





IEEE 754 Floating Point

Advantages of IEEE 754

1. Standard representation for floating-point numbers.
2. Supports both positive and negative real numbers.
3. High precision for scientific calculations.
4. Portable across different computer systems.
5. Efficient hardware implementation.
6. Widely supported by modern processors.

Disadvantages

1. Some decimal numbers cannot be represented exactly.
2. Rounding errors may occur.
3. Precision loss is possible after many calculations.
4. Floating-point operations are slower than integer operations.

Precision Issues

1. Rounding Error

Occurs when the number cannot be represented exactly within the available mantissa bits.

Example:

0.1 cannot be represented exactly in binary.

2. Precision Loss

Very small values may be lost when added to very large numbers.

Example:

$1000000.0 + 0.000001$

The smaller value may not affect the result because of limited precision.

3. Overflow

Occurs when the result exceeds the maximum representable floating-point value.

4. Underflow

Occurs when the result is too small to be represented in normalized form.

Applications

- Scientific Computing
- Artificial Intelligence
- Machine Learning
- Graphics Processing (GPU)
- Weather Forecasting
- Computer Simulation
- Image Processing

- Signal Processing
- Aerospace Engineering
- Financial Analysis

Observation

- IEEE 754 represents floating-point numbers using **Sign**, **Exponent**, and **Mantissa** fields.
- Single precision uses **32 bits**, while double precision uses **64 bits**, providing higher accuracy.
- Floating-point addition requires exponent alignment, mantissa addition, normalization, and rounding.
- Although IEEE 754 provides high precision, some decimal numbers cannot be represented exactly, leading to rounding and precision loss.

Comparison Between Single and Double Precision

Feature	Single Precision	Double Precision
Total Bits	32	64
Sign Bits	1	1
Exponent Bits	8	11
Mantissa Bits	23	52
Bias	127	1023
Precision	About 7 decimal digits	About 15–16 decimal digits
Memory Required	4 Bytes	8 Bytes
Accuracy	Lower	Higher
Applications	Graphics, Embedded Systems	Scientific Computing, AI, Engineering

Conclusion

The decimal number **−13.25** is represented in IEEE 754 format by converting it to binary, normalizing it, determining the **sign bit**, calculating the **biased exponent**, and storing the **mantissa**. In **single precision**, the format consists of **1 sign bit, 8 exponent bits, and 23 mantissa bits**, while **double precision** uses **1 sign bit, 11 exponent bits, and 52 mantissa bits**, offering greater accuracy. During floating-point addition, the processor aligns exponents, adds mantissas, normalizes the result, and rounds it according to IEEE 754 rules. This standardized representation ensures consistent and efficient floating-point arithmetic across modern processors, making it essential for scientific, engineering, graphics, and machine learning applications.

Code of Conduct:

I THIRUMALAPUDI PRANAVI (192411222) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

T.PRANAVI

Signature of the student:

MARKS ALLOCATION

	Problem Understanding (20%)	Method Selection (20%)	Solution Process (25%)	Accuracy of Calculation (20%)	Interpretation of Results (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation